

ui.fullscreen 全面详解

`ui.fullscreen` 是 NiceGUI 框架中基于 Quasar 的 AppFullscreen 插件实现的全屏控制组件，自版本 2.11.0 起新增，提供了进入、退出、切换全屏模式的完整功能，同时支持全屏状态监听、退出行为自定义等扩展能力，适用于需要全屏交互的各类应用场景（如数据可视化、表单编辑、媒体播放等）。

核心特性与重要说明

核心能力

- 支持主动触发全屏（进入、退出、切换）。
- 可配置“长按 ESC 键退出”规则，防止误操作。
- 能监听全屏状态变化并触发回调。
- 支持属性绑定、事件订阅等 NiceGUI 通用组件能力。

关键限制（安全与兼容性）

- 触发限制：**出于浏览器安全策略，全屏模式只能通过用户主动交互触发（如按钮点击），无法通过代码自动触发。
- 浏览器兼容性：**“长按 ESC 退出”功能仅在部分浏览器（如 Google Chrome、Microsoft Edge）中生效，其他浏览器会忽略该配置，按默认行为（单击 ESC 退出）执行。

初始化参数

参数名	类型	说明
<code>require_escape_hold</code>	bool	可选，默认未启用。是否要求用户长按 ESC 键才能退出全屏模式，仅部分浏览器支持。
<code>on_value_change</code>	Callable	可选。全屏状态变化时触发的回调函数，接收 <code>ValueChangeEventArguments</code> 参数，通过 <code>e.value</code> 可获取当前状态（ <code>True</code> 为全屏， <code>False</code> 为非全屏）。

核心属性

`ui.fullscreen` 继承了 NiceGUI 基础组件的通用属性，同时包含以下专属属性：

属性名	类型	可设置性	说明
<code>require_escape_hold</code>	bool	是	控制“长按 ESC 退出”功能的启用状态，动态修改后即时生效（仅支持的浏览器）。
<code>value</code>	BindableProperty	是	绑定全屏状态（ <code>True/False</code> ），支持双向绑定。

属性名	类型	可设置性	说明
<code>visible</code>	<code>BindableProperty</code>	是	控制组件自身的可见性（不影响全屏功能本身）。
<code>classes</code>	<code>Classes[Self]</code>	是	组件的 HTML 类名，用于自定义样式（基于 Tailwind/Quasar）。
<code>style</code>	<code>Style[Self]</code>	是	组件的内联 CSS 样式。
<code>html_id</code>	<code>str</code>	是	组件在 HTML DOM 中的唯一 ID（版本 2.16.0 新增）。
<code>is_deleted</code>	<code>bool</code>	否	只读，标识组件是否已被删除。
<code>client</code>	<code>Client</code>	否	只读，组件所属的客户端实例。

核心方法

全屏控制方法

方法名	参数	返回值	说明
<code>enter()</code>	无	<code>None</code>	触发进入全屏模式（需用户交互触发）。
<code>exit()</code>	无	<code>None</code>	触发退出全屏模式。
<code>toggle()</code>	无	<code>None</code>	切换全屏状态（当前非全屏则进入，当前全屏则退出）。

事件与绑定方法

方法名	核心参数	说明
<code>on_value_change(callback)</code>	<code>callback</code> : 状态变化回调函数	为全屏状态变化添加回调（替代初始化时的 <code>on_value_change</code> 参数）。
<code>bind_value(target_object, target_name='value')</code>	<code>target_object</code> : 绑定目标对象; <code>target_name</code> : 目标属性名	双向绑定全屏状态到目标对象的属性（如组件值变化时同步目标属性，反之亦然）。
<code>bind_value_from(target_object, target_name='value')</code>	同 <code>bind_value</code>	单向绑定：从目标对象属性同步全屏状态到当前组件。
<code>bind_value_to(target_object, target_name='value')</code>	同 <code>bind_value</code>	单向绑定：从当前组件同步全屏状态到目标对象属性。
<code>bind_visibility(...)</code>	目标对象、属性名等	绑定组件可见性到目标对象的属性（支持双向 / 单向绑定）。

通用组件方法

方法名	说明
<code>delete()</code>	删除组件及所有子元素。
<code>update()</code>	同步组件状态到客户端（刷新 UI）。
<code>tooltip(text)</code>	为组件添加悬浮提示文本。
<code>add_resource(path)</code>	为组件添加资源文件（如 CSS/JS）。
<code>mark(*markers)</code>	为组件添加标记，用于测试或查询。

典型使用场景

1. 基础全屏控制（进入 / 退出 / 切换）

通过按钮触发全屏的核心操作，适用于大多数基础场景：

```
from nicegui import ui

# 创建全屏控制实例
fullscreen = ui.fullscreen()

# 按钮触发对应操作
ui.button('进入全屏', on_click=fullscreen.enter)
ui.button('退出全屏', on_click=fullscreen.exit)
ui.button('切换全屏', on_click=fullscreen.toggle)

ui.run()
```

2. 启用“长按 ESC 退出”（防止误操作）

适用于表单编辑、数据录入等需避免意外退出全屏的场景：

```
from nicegui import ui

fullscreen = ui.fullscreen()

# 开关控制是否启用“长按 ESC 退出”
ui.switch('需要长按 ESC 退出').bind_value_to(fullscreen, 'require_escape_hold')
ui.button('切换全屏', on_click=fullscreen.toggle)

ui.run()
```

注：仅 Chrome、Edge 等浏览器支持该功能，其他浏览器开关无效果。

3. 监听全屏状态变化

实时反馈全屏状态（如显示提示、更新文本）：

```
from nicegui import ui

# 初始化时绑定状态变化回调
fullscreen = ui.fullscreen(
    on_value_change=lambda e: ui.notify('进入全屏' if e.value else '退出全屏')
)

ui.button('切换全屏', on_click=fullscreen.toggle)

# 绑定状态到文本标签，实时显示当前模式
ui.label().bind_text_from(
    fullscreen, 'value',
    lambda state: '当前状态: 全屏模式' if state else '当前状态: 普通模式'
)

ui.run()
```

4. 状态双向绑定

将全屏状态与其他组件联动（如开关控制全屏）：

```
from nicegui import ui

fullscreen = ui.fullscreen()

# 开关与全屏状态双向绑定：开关切换 → 全屏切换，手动切换全屏 → 开关同步
ui.switch('启用全屏').bind_value(fullscreen, 'value')

ui.run()
```

注意事项

- 触发限制：**所有进入全屏的操作（`enter()`、`toggle()` 切换至全屏）必须由用户交互触发（如按钮点击、点击事件回调），否则浏览器会拒绝执行。
- 浏览器兼容性：**除“长按 ESC 退出”外，核心全屏功能（进入 / 退出 / 切换 / 状态监听）在主流浏览器（Chrome、Edge、Firefox、Safari）中均支持，但部分浏览器可能存在细微差异（如退出全屏的默认行为）。
- 组件生命周期：**删除组件（`delete()`）后，全屏控制功能将失效，需重新创建实例。
- 样式隔离：**全屏模式下，默认是整个页面全屏，若需指定某个容器全屏，需结合 Quasar 组件或自定义 DOM 元素（`ui.fullscreen` 本身不支持指定容器，需依赖底层 API 扩展）。

版本变更记录

- 2.11.0: 新增 `ui.fullscreen` 组件，支持基础全屏控制和状态监听。
- 2.16.0: 新增 `html_id` 属性。

- 2.18.0: 更新 `on()` 方法, 支持同时指定 Python 回调和 JavaScript 回调。
- 3.0.0: `bind_*` 系列方法新增 `strict` 参数, 支持属性存在性检查。